Two-dimensional early exit optimisation of LLM inference

Jan Hůla[1] David Adamczyk[1] Tomáš Filip[1] Martin Pavlíček[1*] Petr Sosík[1,2]

[1]Institute for Research and Applications of Fuzzy Modelling,
University of Ostrava, 70200 Ostrava, Czech Republic

[2]Institute of Computer Science, Faculty of Philosophy and Science,
Silesian University in Opava, 74601 Opava, Czech Republic

{jan.hula, david.adamczyk, tomas.filip, martin.pavlicek, petr.sosik}@osu.cz

Abstract

We introduce a two-dimensional (2D) early exit strategy that coordinates layer-wise and sentence-wise exiting for classification tasks in large language models. By processing input incrementally sentence-by-sentence while progressively activating deeper layers, our method achieves multiplicative computational savings that exceed those from optimizing either dimension independently. Experimental evaluation across four state-of-the-art LLMs (Llama 3.1, Llama 3.2, Gemma, Qwen; 3B-8B parameters) on three sentiment classification datasets demonstrates additional speed-ups of 1.4–2.3× over optimal layer-wise early exit for simpler tasks with vanilla models, with graceful degradation on complex multi-class problems. Fine-tuning reduces but does not eliminate this advantage. The approach is model-agnostic, requires only lightweight classification adapters, and is orthogonal to complementary efficiency methods such as quantization and pruning. Our findings indicate that 2D early exit strategies excel when semantic information accumulates predictably across input structure, suggesting possible applicability to sequence-processing tasks beyond sentiment classification.
Keywords: Large language model, Inference optimisation, Early exit, Input trimming

1 Introduction

Large Language Models (LLMs) have revolutionised Natural Language Processing (NLP) with their remarkable capabilities across a range of tasks. However, their extraordinary performance comes at a significant computational cost in terms of processing power, memory, and energy. This applies especially to the inference regime, which is run extensively once the model is trained, while training is a very demanding but usually one-time task. For relatively simpler but high-throughput tasks such as classification, deploying full-scale LLMs represents an immense computational overhead. To this end, early exit strategies have emerged as a promising paradigm. They enable a model to terminate its computation at an intermediate layer when it is sufficiently confident in its prediction, while striving to maintain high accuracy.

In this paper, we introduce a two-dimensional (2D) early exit optimisation strategy that combines layer-wise and sentence-wise early exit. The intuition behind it is that, in many classification tasks, the proper class can already be determined from an initial part of the text. To preview the anatomy of the 2D EE strategy, Figure 1 demonstrates how the probability of correct classification progresses along sentences and layers during document processing. Taking a sentence as a natural semantic unit, we split an input sample into sentences and feed an LLM input with sentences one-by-one. Simultaneously, with each new sentence we increment the number of layers

*Corresponding author

processing the so-far read part of the input. This results in rectangular processing blocks growing in 2D (layers vs. sentences) illustrated in Fig. 2. The classification halts once the accumulated confidence (5) exceeds a pre-defined threshold. In this way, a significant speed-up can be achieved compared to standard layer-wise early exit methods. Our findings indicate a possible applicability to a range of sequence-processing tasks, where each input sample can be meaningfully split into a sequence of semantic units that contribute to the overall meaning in an incremental manner. The main contributions are:

- **Methodology:** We integrate layer-wise early stopping with sentence-wise input trimming into one synergistic process. By simultaneously coordinating exiting in both dimensions we achieve multiplicative rather than additive computational savings without sacrificing accuracy.
- **Benchmarking:** We evaluate the 2D strategy against optimal layer-wise early exit and LayerSkip across four state-of-the-art LLMs (Llama-3.1-8B, Llama-3.2-3B, Gemma-3n-E4B, Qwen2.5-7B) on three sentiment classification datasets of varying difficulty. The results demonstrate 1.4–2.3× additional speedup (relative to layer-based early exit) for simpler tasks in vanilla models, with graceful degradation on harder problems.
- **Analysis and insights:** We analyse how fine-tuning affects the 2D advantage, provide hyperparameter optimisation guidance, and identify overall conditions for 2D strategies to excel.

The rest of the paper is structured as follows. Section 2 reviews related work. Section 3 outlines the methodology used, and Section 4 resumes and analyses the experimental results. Section 6 discusses the implications of the work, and Section 7 concludes the paper with directions for future extensions.

2 Background

Traditional methods to improve inference efficiency in LLMs include pruning [18], weight quantisation [3], and knowledge distillation [10]. These techniques compress the model statically; that is, the resulting model operates with fixed structure regardless of input. Although effective, they often cause accuracy degradation and may overfit to specific datasets or domains, reducing transferability between tasks. In contrast, dynamic inference methods adapt computation at runtime. A classical dynamic approach is to employ a *cascade* of models with increasing complexity, where a lightweight model handles easy instances, while only harder ones are passed to a larger model [1]. The *Matryoshka* paradigm [12] generalises this idea into a cascade of progressively complex nested models. MatFormer [4] integrates feedforward network blocks within transformer layers, allowing the extraction of multiple smaller models from one training process at no additional cost.

2.1 Early exit strategy

Early exit methods reduce inference cost by terminating the computation in intermediate layers once a predefined criterion (the *exit trigger*) is met. They are complementary to pruning, quantisation, or distillation and are applicable to virtually any transformer model without fundamental architectural change. The foundational paper [24] introduced the early exit paradigm for deep neural networks. For an overview of the topic, we refer, e.g., to the survey [13] or [19].

Confidence-based exits. Early works attach auxiliary classifiers to intermediate layers and allow inference to stop once confidence exceeds a threshold. DeeBERT [25] pioneered entropy-based exits (off ramps) for BERT, achieving up to 40% inference time reduction on GLUE dataset benchmarks with a drop in accuracy $< 4\%$. Schwartz et al. [22] proposed calibrated confidence thresholds, showing that multi-exit BERTs could achieve 3–5× faster inference with almost the

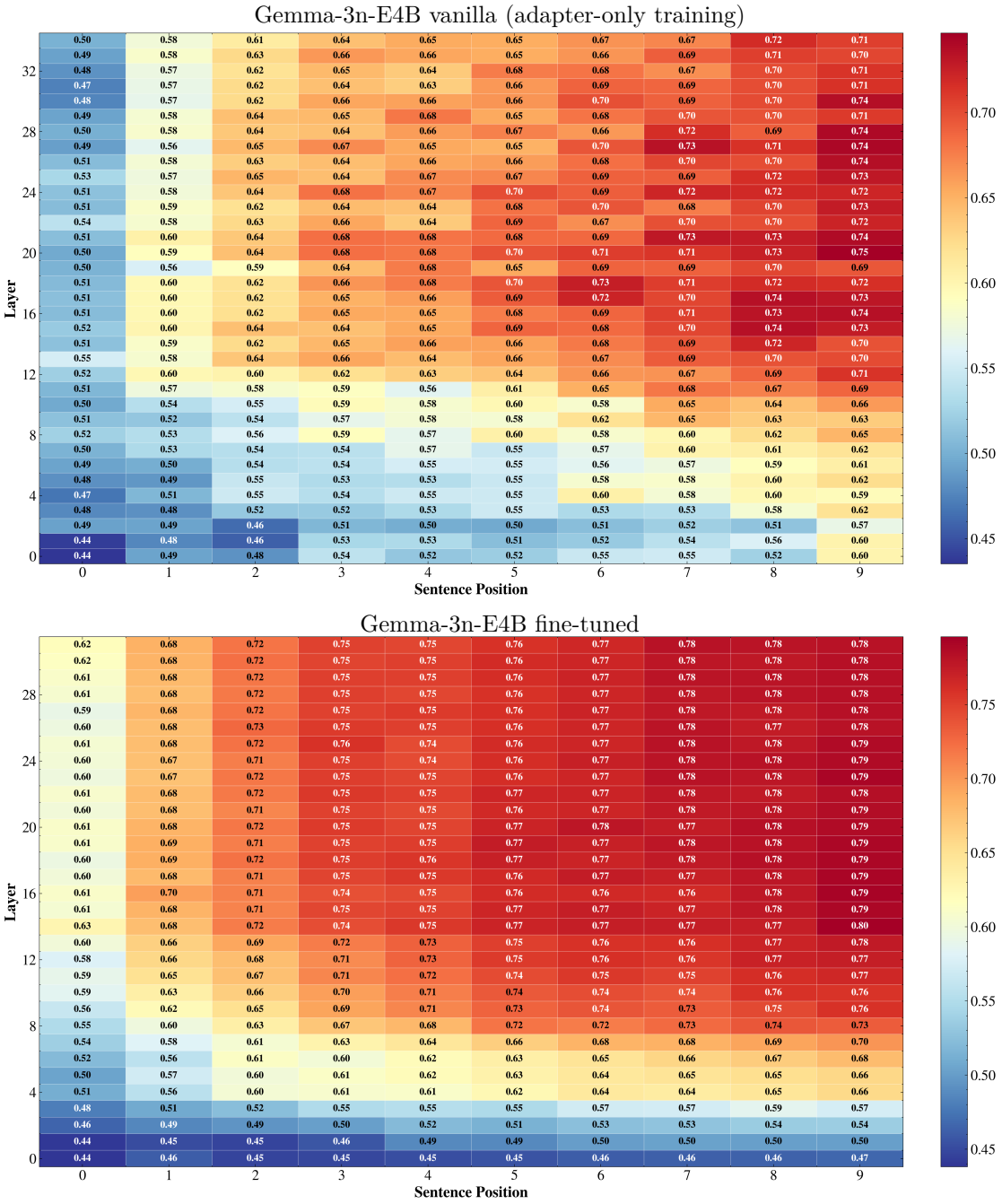

Figure 1: Layer-wise and sentence-wise accuracy of correct classification of Gemma-3n-E4B vanilla (top) and fine-tuned (bottom) on the MMS dataset. Heatmaps based on 7100 samples of 10 sentences each.

same accuracy. To strengthen shallow exits, FastBERT [17] applied self-distillation and adaptable inference, producing reliable intermediate predictions and allowing tunable 1–12× speed-ups with graceful accuracy degradation. PABEE [28] introduces the exit strategy based on an unchanged prediction of internal classifiers for a predefined number of consecutive steps.

Learned exit policies. Rather than fixed heuristics, some approaches learn when to exit. Li et al. [14] introduced a predictive engine for early exiting decisions that allows the network to forecast when it should exit, adapting to computational and energy constraints in real-time. ConsistEE [27] employs reinforcement learning to align training-time supervision with inference-time exits, reporting double digit percentage layer depth reductions with the same or better accuracy. Schuster et al. [20] proposed Confident Adaptive Transformers (CAT), combining additional prediction heads with meta consistency classifiers to guarantee confidence, while still obtaining 1.8–2.7× acceleration. Fan et al. [7] uses statistical classifiers such as SVM trained on easily obtainable token-based features to adaptively decide when to stop inference.

Architecture-integrated approaches. Some methods modify training or architecture to support the exit procedure. LayerSkip [6] introduces layer dropout with progressively higher dropout rates for deeper layers. The early exit loss supervises all intermediate layers using a weighted combination. Self-speculative decoding enables flexible skipping, resulting in 1.82–2.16× speedup on coding, parsing and summarization tasks with marginal accuracy loss. Liu et al. [15] uses a fixed shallow sub-network as a self-draft model with a controlled exit, while the remaining layers serve as the larger target model. GREEN-CODE [8] uses reinforcement learning to optimise exit policy in code generation, demonstrating up to 23–50% energy savings in Llama-3.2-3B and OPT 2.7B. Chen et al. [2] introduces a variety of optimisations targeted both in the training and inference phase, enabling early-exit LLMs to scale up to 30B parameters.

Token-level and sequence-level extensions. Many works, including some already cited [6, 17], combine layer-wise early exit with token-wise exiting, where different tokens exit at different layers. At the sequence level, token-wise halting was pioneered by the Depth-Adaptive Transformer [5], which achieved faster decoding in machine translation. CALM (Confident Adaptive Language Modeling) [21] introduced dynamic allocation of different amounts of compute per input and achieved up to 3× speed-up on generative tasks. Khanna et al. [11] presented a model-agnostic inference framework that reduces computation via dynamic token halting, combined with KV-cache skipping and token-level redundancy reduction, achieving up to 40% resource reduction on frozen models. Recent work extends early exit to reasoning tasks: DEER [26] monitors model confidence at CoT transition points to terminate reasoning sequences early (19–80% reduction in CoT length), while NEAT [16] monitors neuron-level activation dynamics for fine-grained exit decisions (22–28% token reduction across CoT reasoning benchmarks).

Our 2D early exit strategy shares with CALM [21] and token-halting approaches [5, 11] the intuition that not every input unit warrants equal computational depth. However, these methods allocate variable compute per token within a fixed context, requiring careful handling of absent hidden representations and KV-cache management. By contrast, operating at the sentence level – a natural semantic boundary – ensures well-defined embeddings at every active layer, avoiding these complications and leveraging the predictable accumulation of sentiment signal across sentences. Furthermore, by coordinating sentence-wise input trimming with layer-wise early stopping within a unified framework, we achieve multiplicative rather than additive resource reductions in classification tasks.

3 Methods

The core idea of 2D early exit in transformers is to combine layer-wise and token-wise exiting into one framework that acts simultaneously in both dimensions to obtain a synergistic effect. We have chosen a sentence as the elementary input semantic unit. Each sample in the dataset is

split into sentences by the algorithm detailed in the Appendix A. Throughout the paper, *sentence embedding* is defined as the mean of the embeddings of all tokens in the sentence. Each tested model is supplemented with lightweight classification adapters (or classifiers, for short), one per layer, fed with the output embedding of that layer. The classifier architecture is a two-layer fully connected network:

- `Linear(embedding_dim` $\rightarrow$ `256) + RelU`
- `Linear(256` $\rightarrow$ `num_classes) + Softmax`

3.1 Training

The loss function used for training is based on that used in [6, 25, 28] that aims at accurate predictions at intermediate layers. Simultaneously, the loss function was expanded to promote correct classification based on a partial input. Consider a model with L transformer layers. For a sample (x, y) consisting of m sentences, denote by $e_{i,k}(x)$ the embedding of sentence k, $0 \le k \le m-1$, of sample x in the output of layer i, $0 \le i \le L-1$.

Let us define the *prefix embedding* $pe_{i,j}(x)$ as the mean embedding of the first j sentences of the sample,

$$pe_{i,j}(x) = \sum_{k=0}^{j-1} \frac{e_{i,k}(x)}{j}, \quad 0 \le j \le m-1. \tag{1}$$

Two variants of training have been tested: (i) adapter-only training and (ii) fine-tuning of the whole model.

Adapter-only training

In this variant, only the classifiers for individual layers are trained, while the model weights remain frozen. For a training sample (x, y) of m sentences, each classifier is optimised to predict the target label y solely based on the output embedding of the corresponding layer. The sentence embeddings are aggregated using the mean operation and finally processed by the classification adapter, resulting in the loss function

$$\mathcal{L}_i(x, y) = \mathrm{CE}\Big(y, \mathrm{fc}_i\Big(\sum_{j=0}^{m-1} \frac{e_{i,j}(x)}{m}\Big)\Big), \tag{2}$$

where $\mathrm{fc}_i(\cdot)$ is the output of the i-th classification adapter (class probability distribution) and $\mathrm{CE}(\cdot, \cdot)$ is the cross-entropy function.

Fine-tuning

The fine tuning of all tested models, including the classifiers, is implemented with the PEFT adapter. The aggregate loss function

$$\mathcal{L}(x, y) = \sum_{i=0}^{L-1} \lambda_i \ \mathrm{CE}\Big(y, \mathrm{fc}_i\Big(\sum_{j=0}^{m-1} \frac{pe_{i,j}(x)}{m}\Big)\Big) \tag{3}$$

for a sample (x, y) is a weighted average of layer losses. Each layer loss is based on the output of the layer classifier, where its input is an average of all prefix embeddings (1) for this layer and sample. The weights λ_i were experimentally set to 0.8 for the last layer $L-1$ and 0.1 for all previous layers. These values provided the best accuracy across the three tested datasets; however, they may be further auto-tuned for different datasets.

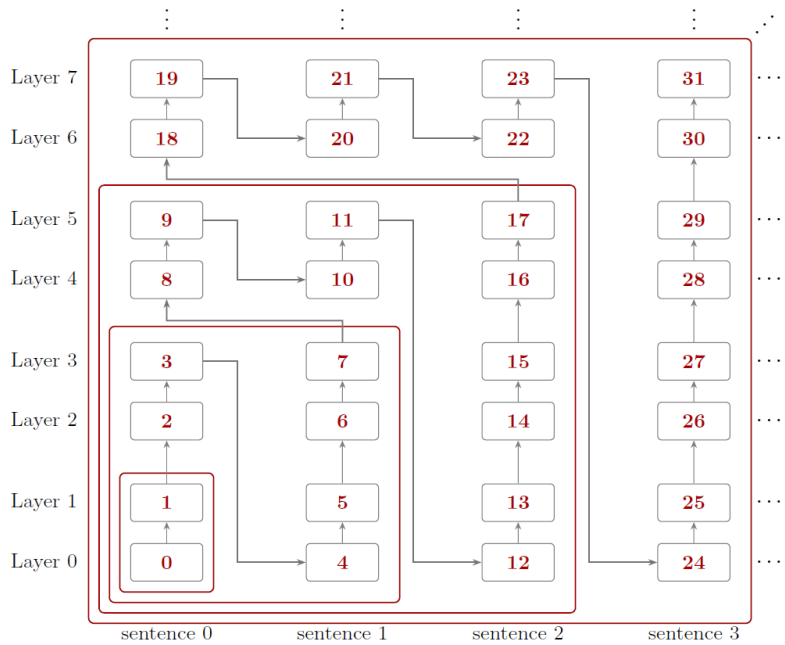

Figure 2: Visualization of the 2D early exit strategy for step size $\Delta = 2$, see eq. (4). Arrows Red frames enclose inference progression blocks corresponding to individual sentences.

3.2 Inference

The key innovation of the 2D early exit strategy, detailed in Algorithm 1, is the two-dimensional progression: the algorithm processes each input sample sentence-by-sentence, with each new sentence progressively increasing the number of available network layers for inference. For an input sample with m sentences, when processing a sentence indexed with s, $0 \leq s \leq m-1$, the algorithm activates the first L_s layers of the model,

$$L_s = \min((s+1) \times \Delta, L), \text{ where } \Delta = \max(1, \lfloor L/m \rfloor). \tag{4}$$

All previous sentences $s_1 < s$ pass through newly activated layers they have not yet passed through, i.e., from L_{s-1} to $L_s - 1$, while sentence s passes through all layers from 0 to $L_s - 1$, resulting in red-framed blocks illustrated in Fig. 2. Hence, the layer depth is coupled to the sentence index, so each new sentence simultaneously expands both contextual breadth and representational dept

Early stopping is controlled by two hyperparameters: ignore threshold τ_{ignore} and acceptance threshold τ_{acc}. Denote by $\text{fc}_{\ell,c}(s_1)$ the softmax output (probability) of the classification adapter in layer ℓ for the input s_1, corresponding to class c. For each ℓ and s_1, the algorithm computes *confidence* as the difference between the maximum and second-maximum softmax outputs:

$$\text{confidence} = \max_c\{\text{fc}_{\ell,c}(s_1)\} - \text{second_}\max_c\{\text{fc}_{\ell,c}(s_1)\}. \tag{5}$$

If confidence exceeds τ_{ignore}, it is added to the accumulated confidence score of the predicted class. When the accumulated confidence of any class exceeds τ_{acc}, the inference terminates immediately, returning the prediction with the highest score. If no class reaches the τ_{acc} threshold after processing all layers and sentences, the algorithm returns the prediction of the final layer for the last sentence.

The 2D early exit inference pseudocode

Input: *example* : input text
$\tau_{ignore}, \tau_{acc}$: hyperparameters – ignore and acceptance threshold
Output: *predicted_label* : prediction for the input example
operations_used : consumed abstract operations to measure speed-up

1. Initialization:

```
sentences ← list of sentences in example
num_sentences ← number of sentences in example
num_layers ← number of layers of the model
operations_used ← 0 (counter of operations used)
Δ ← max(1, num_layers // num_sentences) (step size in layers)
acc ← [0, 0, 0, ...] (confidence accumulators for each class)
```

2. Process sentences and layers till early stopping:

```
for s = 0 to num_sentences − 1 do
    // Process all sentences up to s
    for s_1 = 0 to s do
        // The number of active layers for sentences up to s
        layers_to_traverse = min((s + 1) × Δ, num_layers)

        // Sentences before s pass only newly added chunk of layers
        if s_1 = s then start_layer = 0
                   else start_layer = Δ * s

        for l = start_layer to layers_to_traverse − 1 do
            operations_used ← operations_used + 1

            // Calculate embeddings for the actual sentence using already stored
            // embeddings for previous layers and sentences
            embeddings[l][s_1] ← calculate_embedding(l, sentences[s_1])

            // Classifier at layer l calculates probabilities for all classes
            [p_0, p_1, p_2, ...] ← classifier_l(embeddings[l][s_1])
            predicted_label ← arg max([p_0, p_1, p_2, ...])

            // Calculate confidence of the winning class (max - second max)
            sorted_probs ← SORT([p_0, p_1, p_2, ...]) descending
            confidence ← sorted_probs[0] − sorted_probs[1]

            // Accumulate confidence, filter by ignore threshold
            if confidence > τ_ignore then
                acc[predicted_label] ← acc[predicted_label] + confidence

                // Early stopping check
                if acc[predicted_label] > τ_acc then
                    return (predicted_label, operations_used)
```

3. All layers and sentences processed – no early stopping:

```
final_label ← prediction from the last sentence at last layer
return (final_label, num_layers × num_sentences)
```

Algorithm 1: The 2D early exit inference for one input example.

3.3 Metrics and evaluation

The experiments were evaluated using two metrics: (i) classification accuracy and (ii) speed-up enabled by early exit, expressed as the ratio of abstract operations in a full and early terminated inference. For each model, dataset, and training mode, let $acc(\ell)$ be the accuracy of the classifiers at layer ℓ on the test split. We set the acceptable *accuracy threshold* to

$$acc_thr = \max_{0 \le \ell < L} \{acc(\ell)\} - T, \tag{6}$$

where T is the allowed accuracy loss. We repeated the experiments for $T = 0.02$ (2%) and $T = 0.04$ (4%). Then, for all EE methods tested, we ran the inference until the accuracy threshold was met and calculated the corresponding speed-up. For layer-wise early exit, the speed-up was set as

$$\textit{speed-up} = L/(L_e + 1), \tag{7}$$

where L is the total number of transformer layers in the model and L_e is the exit layer at which the accuracy threshold is met (layers numbered from 0 to $L - 1$).

The speed-up achieved by the 2D EE strategy was quantified as follows. We count the processing of one sentence in one transformer layer as one abstract operation. When the accuracy threshold is not met, Algorithm 1 processes the complete input through all layers using $m \times L$ operations, which corresponds to an inference with no early exit. When early stopping occurs, the algorithm returns the number of *operations_used*, and the speed-up is calculated as

$$\textit{speed-up} = (m \times L)/\textit{operations_used}. \tag{8}$$

One can argue that, since each sentence attends to all previous sentences, its processing time is proportional to its order in the input text. However, this is true only for long inputs of at least thousands of sentences. Denote

tps	average number of tokens per sentence (≈ 15 for user reviews);
embed_dim	embedding dimension of the model;
exp_f	expansion factor in the MLP module in the transformer layer.

Considering, e.g., Llama-3.2-3B with $embed_dim = 3072$ and $exp_f \approx 2.67$, computationally intensive operations in a transformer layer processing the s-th sentence of an input example, are:

Query, key, value computation:	$\approx 3 \times tps \times embed_dim^2 \approx 4.2 \times 10^8$
Attention matrix computation:	$\approx s \times tps^2 \times embed_dim \approx 6.9 \times 10^5 \times s$
MLP expansion and compression:	$\approx 2 \times tps \times embed_dim^2 \times exp_f \approx 7.6 \times 10^8$

Therefore, the linear term containing s becomes relevant for $s = \Omega(10^3)$.

4 Experiments

The aim of the experiments was to compare the efficiency of the 2D EE strategy with recent approaches based on exiting at intermediate layers. Numerous studies, such as [6, 7, 14, 15, 20, 25, 29], and many others, have examined methods of triggering layer-wise early exit. They often differ in language models and datasets, and some use additional mechanisms to improve early exit efficiency, which makes comparison difficult or impossible. Therefore, we compare our 2D EE strategy against *the optimal EE trigger* with layer-wise speed-up calculated by formulas (6) and (7). This baseline provides the *supremum speed-up* of all possible layer-wise exit triggers, implying that the additional speed-up achieved with the 2D EE strategy is guaranteed with respect to a range of previous layer-wise EE studies.

The experiments were conducted in two training modes described in Section 3.1. For each mode, we compared the results of the optimal layer-wise early exit described above with those of the 2D early exit. Finally, we also conducted experiments that compare the 2D EE results with the LayerSkip method [6] for the Llama-3.1-8B and Llama-3.2-3B models (as the LayerSkip library is not available for Gemma nor Qwen).

4.1 Datasets

We used three publicly available datasets for sentiment analysis.

1. Steam gaming Reviews[1] (*Steam*), two classes.
2. The Massive Multilingual Sentiment Corpora[2] (*MMS*) of reviews in multiple languages and topics, three classes.
3. Amazon Reviews for SA fine-grained[3] (*Amazon-5*), Amazon product reviews, five classes.

From each dataset, we randomly chose a sentiment-balanced English subset of 15K samples (10K train, 5K test, English only) restricted to samples with at least 10 sentences.

4.2 Language models

ModernBERT is the encoder-only model released in 2024, and we use it as a baseline for our experiments. The Llama, Gemma, and Qwen families are popular and widely used medium-sized decoder-only LLM. We have chosen four of these models, sized from 3B to 8B parameters.

Model name	**Layers**	**Model card**
ModernBERT Large	28	https://huggingface.co/answerdotai/ModernBERT-large
Llama-3.1-8B Instruct	32	https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct
Llama-3.2-3B Instruct	28	https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct
Gemma-3n-E4B	35	https://huggingface.co/google/gemma-3n-E4B-it
Qwen2.5-7B	28	https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

4.3 Implementation

The experiments were conducted on a single-node DGX Station A100 with 1× AMD EPYC 7742 (64 cores / 128 threads), 4× Nvidia A100 SXM4 GPUs with 40 GB of HBM2 each (NVLink-connected), and approximately 512 GB of system RAM. We used the same training setup for all models. The training of one adapter took 2.7 min and, as the adapters are independent, all can be trained in parallel. We used the following setting:

- Optimiser: Adam with a learning rate 1×10^{-5}
- Batch size: 128
- Epochs: 20

For fine-tuning, including ModernBERT and LayerSkip implementations, the LoRA configuration included a rank of 64 and a scaling factor (alpha) of 16, resulting in a training time (Llamma-3.1-8B) of 66.9 min:

- Optimiser: AdamW with a learning rate 1×10^{-4}

[1] `https://www.kaggle.com/datasets/andrewmvd/steam-reviews`
[2] `https://huggingface.co/datasets/Brand24/mms`
[3] `https://www.kaggle.com/datasets/yacharki/amazon-reviews-for-sentianalysis-finegrained-csv`

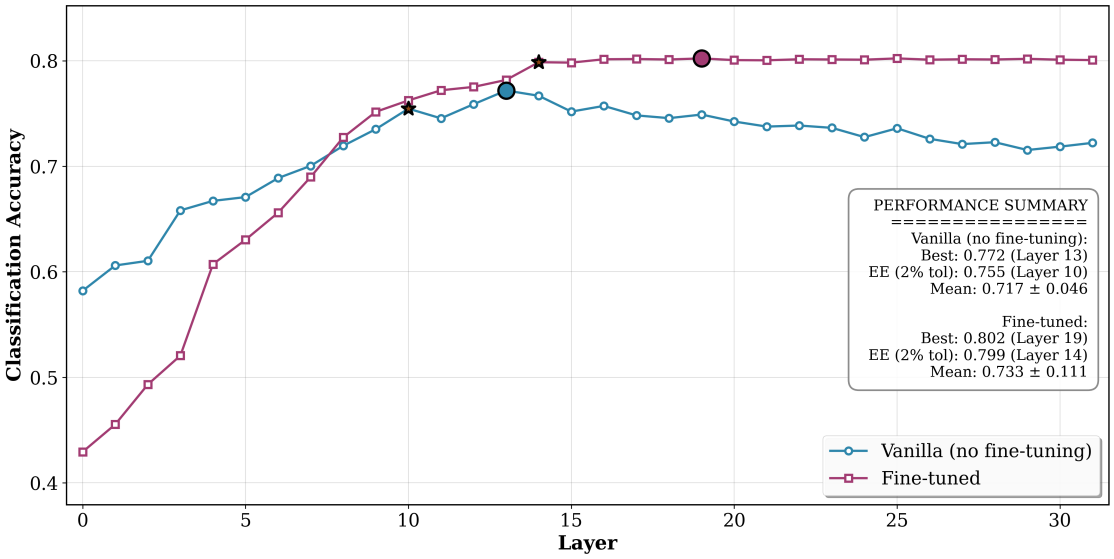

Figure 3: Layer-wise accuracy of LLama-3.1-8B, vanilla model (above) and fine-tuned model (bellow), averaged on the MMS test dataset. Full circles mark the layers with the best accuracy. Stars mark the first layers with the accuracy 2% below maximum.

- Batch size: 16
- Epochs: 2

For the layer-wise early exit, the model was fed the whole input example, and classifiers at each layer calculated their predictions from the embedding of the last sentence of the example (which aggregated information from the entire example thanks to the attention mechanism).

The 2D early exit inference was implemented by Algorithm 1, controlled by two hyperparameters τ_{ignore} and τ_{acc}. The algorithm is designed for downstream task inference, where the hyperparameters are already pre-set. A systematic exploration of hyperparameter space ($\approx$ 250 settings) was needed to explain the behaviour of the algorithm for all 24 combinations of dataset/model/training_mode – see Fig. 6 for an example. To optimise time and energy, we pre-computed and stored embeddings for each sentence of all testing samples and all layers of each model, along with the corresponding metadata. The embeddings were used to compute the probability vectors $[p_0, p_1, p_2, \ldots]$ in the output of the trained classifiers, which were then processed by Algorithm 1. This sped-up the experiments by $\approx$ two orders of magnitude.

5 Results

We compared the efficiency of the 2D early exit strategy with other EE methods tested, based on the metrics detailed in Section 3.3. Generally, the 2D early exit strategy achieved a substantial speed-up increase across different model architectures for simpler tasks, while its results were inconclusive for the most complex Amazon-5 dataset. The 2D EE advantage depends on the task complexity, the training protocol (adapter-only training versus model-fine-tuning), and the allowed accuracy loss, as detailed in Sec. 5.5.

Model	ModernBERT	Llama-3.1	Llama-3.2	Gemma-3n	Qwen2.5
Dataset	Large	8B	3B	E4B	7B
Steam	0.87	0.97	0.96	0.96	0.96
MMS	0.78	0.80	0.79	0.78	0.79
Amazon-5	0.62	0.65	0.63	0.63	0.64

Table 1: Accuracy of the fine-tuned ModernBERT Large versus other models tested.

5.1 Baseline results

To put results into context with numerous studies using BERT-based models, we compared the accuracy of ModernBERT Large with the mid-size LLMs used in our 2D EE study. The results of the fine-tuned models in the three sentiment classification datasets are listed in Table 1. No early exit was used.

As an early exit speed-up baseline, we use the optimal idealised layer-wise early exit. Fig. 3 shows the typical layer-wise accuracy curves for vanilla and fine-tuned LLMs (no early exit). In vanilla models, the layer-wise accuracy usually reaches its maximum approximately halfway through the model depth and then slightly decreases. Fine-tuning increases the accuracy in all models as expected, but also flattens the layer-wise accuracy curves in the upper layers. To view this phenomenon from another perspective, Fig. 5 shows the cosine distances between embeddings in consecutive layers of Llama-3.1-8B.

5.2 Results of adapter-only training

The results are summarised in Table 2. In some cases marked with *, the 2D EE strategy did not reach the accuracy threshold (6) for a loss of 2%. For simpler classification tasks (Steam with 2 classes and MMS with 3 classes), the 2D approach consistently outperformed the layer-wise early exit by factors of 1.4-2.3× (Steam) and 1.1-2.7× (MMS). For the most challenging dataset (Amazon-5 with 5 classes), 2D EE dominated only for Gemma and Qwen models, providing 1.1-1.5 speed-up improvement, while it was unfavourable for Llama models.

Dataset	**Model**	**Acc**	**Accuracy loss 2%**			**Accuracy loss 4%**		
			layer-wise EE		**2D EE**	**layer-wise EE**		**2D EE**
			Layer	Spd-up	Spd-up	Layer	Spd-up	Spd-up
Steam	Gemma-3n-E4B	0.94	14	2.3	**4.2**	11	2.9	**6.4**
	Llama-3.1-8B	0.95	9	3.2	**5.8**	7	4.0	**9.2**
	Llama-3.2-3B	0.94	9	2.8	**5.3**	7	3.5	**7.5**
	Qwen2.5-7B	0.95	12	2.2	**3.1**	10	2.5	**5.1**
MMS	Gemma-3n-E4B	0.74	14	2.3	**4.6**	12	2.7	**6.2**
	Llama-3.1-8B	0.77	11	2.7	**3.3**	9	3.2	**4.3**
	Llama-3.2-3B	0.76	10	2.5	*	9	2.8	**3.2**
	Qwen2.5-7B	0.76	13	2.0	**3.7**	13	2.0	**5.0**
Amazon-5	Gemma-3n-E4B	0.55	15	2.2	**2.5**	14	2.3	**3.5**
	Llama-3.1-8B	0.61	10	**2.9**	*	9	**3.2**	1.6
	Llama-3.2-3B	0.59	10	**2.5**	*	9	**2.8**	1.6
	Qwen2.5-7B	0.56	14	1.9	**2.1**	13	2.0	**2.9**

Table 2: Results of the 2D early exit strategy in vanilla models. The cases when 2D EE did not reach the 2% accuracy loss threshold are marked by *.

5.3 Results of fine-tuned models

Fine-tuning (Table 3) significantly reduced the advantage of the 2D EE strategy, which remained clearly superior for the simplest Steam dataset by a factor of 1.4-1.7×.. In MMS with 3 classes, the advantage was still 1.05-1.25× for 4% accuracy loss, but almost vanished for 2% loss. Finally, the layer-wise early exit outperformed the 2D EE method on the Amazon-5 dataset for both 2% and 4% accuracy loss. This suggests that, while adapter-only training preserves the layer-wise accuracy variations beneficial for 2D early exit, model fine-tuning flattens these variations (Fig. 1), hence reducing the 2D advantage despite improving overall accuracy. Further analysis is provided in the ablation study, Sect. 6.1.

Dataset	**Model**	**Acc**	**Accuracy loss 2%**			**Accuracy loss 4%**		
			layer-wise EE		**2D EE**	**layer-wise EE**		**2D EE**
			Layer	Spd-up	Spd-up	Layer	Spd-up	Spd-up
Steam	Gemma-3n-E4B	0.96	13	2.5	**3.2**	12	2.7	**4.2**
	Llama-3.1-8B	0.97	9	3.2	**4.3**	7	4.0	**6.9**
	Llama-3.2-3B	0.96	8	3.1	**4.1**	7	3.5	**5.8**
	Qwen2.5-7B	0.96	10	2.5	**3.2**	9	2.8	**4.5**
MMS	Gemma-3n-E4B	0.78	19	1.8	**1.9**	15	2.2	**2.3**
	Llama-3.1-8B	0.80	14	2.1	**2.2**	10	2.9	**3.2**
	Llama-3.2-3B	0.79	12	**2.2**	2.1	10	2.5	**2.9**
	Qwen2.5-7B	0.79	15	1.8	**1.9**	13	2.0	**2.5**
Amazon-5	Gemma-3n-E4B	0.63	18	**1.8**	1.6	14	**2.3**	1.9
	Llama-3.1-8B	0.65	14	**2.1**	1.6	12	**2.5**	2.1
	Llama-3.2-3B	0.63	12	**2.2**	1.6	10	**2.5**	2.1
	Qwen2.5-7B	0.64	14	**1.9**	1.4	14	**1.9**	1.8

Table 3: Results of the 2D early exit strategy in fine-tuned models.

5.4 LayerSkip experiments

The experiments were carried out for the Llama-3.1-8B and Llama-3.2-3B models, which are supported by the available LayerSkip implementation. Unlike the original paper [6], where the exit layer is manually set as one of the hyperparameters, we let LayerSkip pass through all layers and stored the accuracies per layer. Its best accuracy among all layers was in line with our layer-wise EE results for all three datasets, confirming the robustness of the methodology used. For comparison with the 2D EE approach (Tab. 4), we took the first layer in which LayerSkip matched the accuracy threshold (6) and calculated the corresponding speed-up (7). The speed-up of the 2D EE strategy was clearly superior for the Steam and MMS datasets. LayerSkip dominated for the Amazon-5 dataset with an accuracy loss of 2%, but 2D EE matched or outperformed LayerSkip for 4% loss.

5.5 Analysis of results

The advantage of the 2D EE strategy over layer-wise early exit (or the LayerSkip method) in our experiments depends on several factors.

Task complexity

The 2D EE dominated in simpler tasks (Steam and MMS dataset with classification to 2 or 3 classes, respectively), but layer-wise EE proved more efficient for the most challenging task (Amazon-5 with 5 classes), except for two test cases, suggesting that 2D strategies excel when semantic information accumulates predictably across sentences.

Dataset	Model	Acc	Accuracy loss 2%			Accuracy loss 4%		
			LayerSkip		2D EE	LayerSkip		2D EE
			Layer	Spd-up	Spd-up	Layer	Spd-up	Spd-up
Steam	Llama-3.1-8B	0.97	11	2.9	**4.3**	10	3.2	**6.9**
	Llama-3.2-3B	0.96	11	2.5	**4.1**	10	2.8	**5.8**
MMS	Llama-3.1-8B	0.80	15	**2.1**	**2.1**	15	2.1	**3.2**
	Llama-3.2-3B	0.79	15	1.9	**2.1**	15	1.9	**2.9**
Amazon-5	Llama-3.1-8B	0.65	16	**2.0**	1.6	15	**2.1**	**2.1**
	Llama-3.2-3B	0.63	15	**1.9**	1.6	15	1.9	**2.1**

Table 4: Results of LayerSkip compared to the 2D EE technique.

Accuracy loss

The chosen loss of 2% balanced performance preservation with speedup across all tested configurations, while the loss of 4% favoured the 2D EE methods. This is attributed to the fact that 2D EE provides much more possible exit points, resulting in a graceful accuracy decay when cutting both the number of processing layers and the processed part of the input.

Fine-tuning

Fine-tuning consistently decreased the advantage of 2D EE over layer-wise EE across all three datasets and all models tested. We leave a deeper analysis of this phenomenon to the ablation study in Sect. 6.1.

6 Discussion

6.1 Ablation study

To understand the individual contributions of different components in our 2D early exit framework, we conducted an ablation study on the Gemma-3n-E4B model using the MMS dataset. We compared four configurations: (i) vanilla model with adapter-only training and layer-wise early exit, (ii) same model with 2D early exit, (iii) fine-tuned model with layer-wise early exit, and (iv) same model with 2D early exit.

Table 5 shows the results for accuracy loss of 2% . The baseline configuration (i) achieves 2.3× speed-up through layer-wise early exit alone. Adding sentence-wise trimming in configuration (ii) increases the speed-up to 4.6×, demonstrating that the 2D coordination provides a 2× multiplicative benefit beyond layer-wise exiting. However, model fine-tuning (iii) and (iv), while improving overall accuracy (0.78 vs. 0.74), provides a speed-up of 1.8 / 1.9 for layer-wise / 2D early exit, respectively, reducing the multiplicative benefit of 2D EE to 1.06×.

Configuration	Max. accuracy	Speed-up
(i) Adapter + Layer-wise EE	0.74	2.3×
(ii) Adapter + 2D EE	0.74	4.6×
(iii) Fine-tuned + Layer-wise EE	0.78	1.8×
(iii) Fine-tuned + 2D EE	0.78	1.9×

Table 5: Ablation study results on Gemma-3n-E4B with MMS dataset (accuracy loss of 2%).

To analyse the role of fine-tuning, we calculated heatmaps of accuracy along the layer and sentence axis for Gemma-3n-E4B, both vanilla and fine-tuned (1) on an MMS sub-dataset of samples with 10 sentences (7100 samples). Fig. 4 compares the average block-wise accuracy

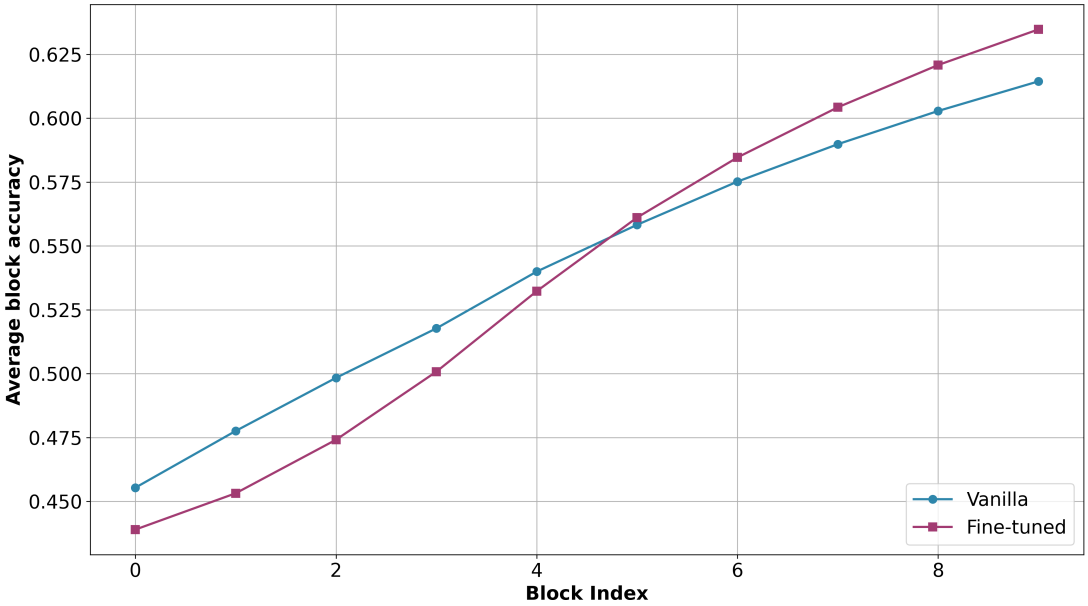

Figure 4: Average block accuracies of 2D early exit for Gemma-3n-E4B on the MMS sub-dataset of samples with 10 sentences.

curves based on these data for 2D EE progression blocks defined in Sect. 3.2. Rather surprisingly, while fine-tuning increased accuracy in later layers and sentences, it simultaneously decreased it in earlier layers and sentences, despite the fact that the fine-tuning loss function (3) progressively penalises loss in early layers and sentences.

6.2 Hyperparameter setting

The hyperparameters used during training include the weights λ_i of the losses of individual layers in the aggregate loss function (3). While [5, 6] use a quadratic increase in weights towards higher layers, we experimentally verified that a simpler setting described in Section 3.1 provided comparable results, indicating the robustness of the weights.

Inference requires tuning two hyperparameters: τ_{ignore} (confidence threshold for accumulation) and τ_{acc} (acceptance threshold for early stopping). Analysis of the hyperparameter space, illustrated in Figure 6, reveals a low sensitivity to hyperparameters: well-behaved optimisation landscapes with broad and clear global maxima, making efficient search strategies viable. A simple hill-climbing or coarse-to-fine grid search can identify near-optimal configurations within 20-30 evaluations. The optimal values tend to cluster predictably: τ_{ignore} typically ranges from 0.3 to 0.5 (filtering low-confidence predictions), while τ_{acc} scales with the complexity of the task.

6.3 Comparison with other works

Our results align with and extend existing findings. DeeBERT achieved $\approx 40\%$ inference reduction in the GLUE SST-2 task with $< 4\%$ accuracy loss, in particular 44-47% reduction in SST-2 with $\approx 2\%$ accuracy loss, comparable to our experiment with Steam dataset (3.2-4.3$\times$ speed-up, i.e., 69-77% reduction, <2% loss). The comparison with LayerSkip showed an advantage of 2D EE $\approx 1.6\times$ for the Steam dataset (2% accuracy loss) and $\approx 1.8\times$ for MMS (4% loss). Unlike methods requiring architectural modifications (LayerSkip, MatFormer) or reinforcement learning

(ConsistEE, GREEN-CODE), our approach remains model-agnostic and training-efficient, requiring only lightweight classification adapters. The key distinction is our simultaneous optimisation across both layer and token dimensions, which -— as demonstrated -— can provide multiplicative rather than additive benefits when task complexity permits early semantic convergence.

6.4 Limitations

Datasets

We used three public datasets of customer reviews with different difficulty (2 / 3 / 5 classes). Consequently, the fine-tuned models returned average test set accuracies of 0.96 / 0.79 / 0.64, with only slight variations between models. This choice of datasets allowed us to demonstrate how 2D EE efficiency depends on the complexity of the task. We could not use some popular benchmarks such as GLUE, since longer inputs ($\geq$10 sentences) are needed to demonstrate the 2D EE strength. Experiments with datasets beyond sentiment classification will be needed to assess general applicability.

Baselines

To avoid comparison with numerous previous works using various training and inference mechanisms, which necessarily leads to methodological inconsistency and incompleteness, we generalised layer-wise EE studies to an *optimal early exit* using an exit layer with the best possible speed-up (Section 4). This allowed for a principal comparison of the 1D (layer-wise) and 2D (layer-wise and token-wise) early exit philosophy. In addition, we included a direct comparison of the results with an influential recent study [6].

Metrics

Early exit trades a substantial speed-up (and energy reduction) for a small output quality drop. The choice of accuracy metrics suffices for this purpose, as in other similar studies. The inference speed-up was expressed as the ratio of abstract operations (processed layers/input sentences) used in the full and the exiting model, which is machine-dependent model-agnostic. We do not include wall-clock timing results because a fair runtime evaluation would require an optimised custom CUDA kernel for the proposed early-exit computation. A naive implementation would not faithfully represent the method's true efficiency, as the measured latency would be dominated by implementation overhead rather than by the underlying reduction in computation. Developing such a kernel is outside the scope of this paper, so we restrict our evaluation to theoretical/computational analysis.

6.5 Orthogonality with other methods

The 2D EE strategy can be combined with a range of complementary methods to further enhance efficiency. One option is to integrate 2D EE with *Mixture of Experts* (MoE) architectures [23], which dynamically route input through a sparse set of specialised sub-networks. *Matryoshka* representation learning [12] organises representations into nested levels of complexity. Among the models we tested, Gemma-3n-E4B uses the Matryoshka architecture, and therefore our results confirm its orthogonality with the 2D EE technique. Similarly, *quantization methods* [3] can allow intermediate exits to operate with low precision when used alongside 2D EE. *Pruning techniques* [18] can be applied to remove redundant weights or neurons. Combined with 2D EE, pruning can further lower computational requirements by simplifying both the early and late stages of the model. Finally, *knowledge distillation* [10] offers a compelling way to train intermediate exit layers. By distilling knowledge from the full model into these layers, one can boost the accuracy of early predictions in both layer-wise and token-wise levels.

7 Conclusion and future work

We have introduced a novel two-dimensional early exit strategy that simultaneously coordinates layer-wise and sentence-wise exiting for classification tasks in large language models. By processing input incrementally sentence-by-sentence while progressively activating deeper layers, our method achieves computational savings that exceed those possible through layer-wise or token-wise optimisation alone.

Our experimental evaluation across four state-of-the-art LLMs (Gemma-3n 4B, Llama-3.1-8B, Llama-3.2-3B, Qwen-2.5 7B) on three sentiment classification datasets demonstrates substantial additional speed-ups against optimal layer-wise early exit with vanilla models: up to 2.3× for both binary classification and a three-class problem, and up to 1.5× for complex multi-class problems (Amazon-5 with 5 classes). The 2D advantage persists, though diminished, also with fine-tuned models (up to 1.7× for two classes and up to 1.3× for three classes). However, the 2D advantage disappears on Llama models for the Amazon-5 task, indicating that 2D strategies excel primarily when semantic information accumulates predictably across sentences.

An intriguing finding is that fine-tuning, while improving overall accuracy, paradoxically reduces the 2D early exit advantage by flattening accuracy variations across early layers and partial inputs. This suggests an unexplored tension between optimising for final accuracy and preserving the gradient structures that enable effective early exiting. The heatmaps in Fig. 1 and the graph in Fig. 4 illustrate this phenomenon, revealing that fine-tuning increases accuracy in later processing stages while simultaneously decreasing it in earlier ones—despite the loss function explicitly encouraging early predictions.

Future work should address several directions. First, developing fine-tuning strategies that explicitly preserve or enhance accuracy gradients across both dimensions could unlock the full potential of 2D early exit. Techniques for stabilising layer-wise semantics, such as GER-steer [9], might be helpful. Second, extending the approach beyond sentiment classification to tasks such as topic categorisation, intent detection, or question answering would test its generalisability. Third, integrating reinforcement learning to enable adaptive, per-sample layer skipping patterns could provide even more flexible resource allocation. Finally, combining 2D early exit with orthogonal techniques such as quantisation, pruning, or mixture-of-experts architectures could yield multiplicative efficiency gains across multiple optimisation dimensions.

Acknowledgements

The authors thank Lou Nováků for carrying out experiments with ModernBERT.

Declarations

Funding

This article was produced with the financial support of the European Union under the REFRESH – Research Excellence For REgion Sustainability and High-tech Industries project number CZ.10.03.01/00/22_003/0000048 via the Operational Programme Just Transition, and under the: Biography of Fake News with a Touch of AI: Dangerous Phenomenon through the Prism of Modern Human Sciences project no.: CZ.02.01.01/00/23_025/0008724 via the Operational Programme Jan Ámos Komenský. It was also supported by the Silesian University in Opava under the Student Funding Plan, project SGS/9/2024.

Conflict of interest

The authors declare that they have no conflict of interest.

Data availability

Project (continuously updated): `https://github.com/irafm-llm/2D_early_exit_inference`
Datasets: `https://huggingface.co/datasets/irafm-llm/2D_early_exit_inference`

References

[1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: how to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[2] Yanxi Chen, Xuchen Pan, Yaliang Li, Bolin Ding, and Jingren Zhou. EE-LLM: large-scale training and inference of early-exit large language models with 3D parallelism. *arXiv preprint arXiv:2312.04916*, 2023.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36: 10088–10115, 2023.

[4] Fnu Devvrit, Sneha Kudugunta, Aditya Kusupati, Tim Dettmers, Kaifeng Chen, Inderjit Dhillon, Yulia Tsvetkov, Hanna Hajishirzi, Sham Kakade, Ali Farhadi, et al. MatFormer: nested transformer for elastic inference. *Advances in Neural Information Processing Systems*, 37:140535–140564, 2024.

[5] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *ICLR 2020: Eighth International Conference on Learning Representations*, pages 1–14, 2020.

[6] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12622–12642. Association for Computational Linguistics, 2024. URL `http://dx.doi.org/10.18653/v1/2024.acl-long.681`.

[7] Siqi Fan, Xin Jiang, Xiang Li, Xuying Meng, Peng Han, Shuo Shang, Aixin Sun, Yequan Wang, and Zhongyuan Wang. Not all layers of LLMs are necessary during inference. *arXiv preprint arXiv:2403.02181*, 2024.

[8] Shashikant Ilager, Lukas Florian Briem, and Ivona Brandic. GREEN-CODE: optimizing energy efficiency in large language models for code generation, 2025. URL `https://arxiv.org/abs/2501.11006`.

[9] Xinyan Jiang, Wenjing Yu, Di Wang, and Lijie Hu. Global evolutionary steering: refining activation steering control via cross-layer consistency, 2026. URL `https://arxiv.org/abs/2603.12298`.

[10] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, 2020.

[11] Danush Khanna, Aditya Kumar Guru, Srivarshinee Sridhar, Zidan Ahmed, Rubhav Bahirwani, Meetu Malhotra, Vinija Jain, Aman Chadha, Amitava Das, and Kripabandhu Ghosh. Quicksilver – speeding up LLM inference through dynamic token halting, KV skipping, contextual token fusion, and adaptive matryoshka quantization, 2025. URL `https://arxiv.org/abs/2506.22396`.

[12] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30233–30249. Curran Associates, Inc., 2022.

[13] Stefanos Laskaridis, Alexandros Kouris, and Nicholas D Lane. Adaptive inference through early-exit networks: Design, challenges and directions. In *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning*, pages 1–6, 2021.

[14] Xiangjie Li, Chenfei Lou, Yuchi Chen, Zhengping Zhu, Yingtao Shen, Yehan Ma, and An Zou. Predictive exit: Prediction of fine-grained early exits for computation-and energy-efficient inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 8657–8665, 2023.

[15] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Kai Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding via double early exiting. *arXiv preprint arXiv:2404.18911*, 2024.

[16] Kang Liu, Yongkang Liu, Xiaocui Yang, Peidong Wang, Wen Zhang, Shi Feng, Yifei Zhang, and Daling Wang. NEAT: neuron-based early exit for large reasoning models. *arXiv preprint arXiv:2602.02010*, 2026.

[17] Weijie Liu, Peng Zhou, Zhe Zhao, Zhiruo Wang, Haotang Deng, and Qi Ju. FastBERT: a self-distilling BERT with adaptive inference time. *arXiv preprint arXiv:2004.02178*, 2020.

[18] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-pruner: on the structural pruning of large language models. *Advances in Neural Information Processing Systems*, 36:21702–21720, 2023.

[19] Haseena Rahmath P, Vishal Srivastava, Kuldeep Chaurasia, Roberto G Pacheco, and Rodrigo S Couto. Early-exit deep neural network: a comprehensive survey. *ACM Computing Surveys*, 57(3):1–37, 2024.

[20] Tal Schuster, Adam Fisch, Tommi Jaakkola, and Regina Barzilay. Consistent accelerated inference via confident adaptive transformers. *arXiv preprint arXiv:2104.08803*, 2021.

[21] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.

[22] Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A Smith. The right tool for the job: Matching model and instance complexities. *arXiv preprint arXiv:2004.07453*, 2020.

[23] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

[24] Surat Teerapittayanon, Bradley McDanel, and Hsiang-Tsung Kung. BranchyNet: fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*, pages 2464–2469. IEEE, 2016.

[25] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. DeeBERT: dynamic early exiting for accelerating BERT inference. *arXiv preprint arXiv:2004.12993*, 2020.

[26] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. Dynamic early exit in reasoning models, 2025. URL https://arxiv.org/abs/2504.15895.

[27] Ziqian Zeng, Yihuai Hong, Hongliang Dai, Huiping Zhuang, and Cen Chen. ConsistentEE: a consistent and hardness-guided early exiting method for accelerating language models inference, 2024. URL https://arxiv.org/abs/2312.11882.

[28] Wangchunshu Zhou, Canwen Xu, Tao Ge, Julian McAuley, Ke Xu, and Furu Wei. BERT loses patience: fast and robust inference with early exit. *Advances in Neural Information Processing Systems*, 33:18330–18341, 2020.

[29] Wei Zhu. LeeBERT: learned early exit for BERT with cross-level optimization. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2968–2980, 2021.

Appendix A Splitting the text into sentences

The datasets contain one user contribution per sample, such as a product review, which must be split into individual sentences. Due to the informal style of user reviews, simple splitting is insufficient, and regular expressions are required. Our solution employs two regular expressions. The first is a negative look-behind, preventing splits at positions that are not genuine sentence boundaries. Specifically, it excludes periods in abbreviations (e.g. "Dr." or "etc.") and multiple consecutive periods ("..."). The second regular expression actively identifies true delimiters – a period followed by whitespace or an exclamation mark or question mark (potentially repeated) followed by whitespace. In practice, consider the review "Excellent product! It is great. I recommend Dr. Smith." The algorithm correctly identifies sentence boundaries after "product!" and "great." but preserves "I recommend Dr. Smith." as a unit since the abbreviation "Dr." satisfies the negative look-behind conditions.

Appendix B Additional explanatory results

Figure 5 shows cosine distances between embeddings of consecutive layers in Llama-3.1-8B model, in both vanilla and fine-tuned version, using the MMS dataset testing split. The local peaks visible between layers 13-15 correspond to the optimal exit points with the 2% accuracy loss in Tables 2 and 3.

Figure 6 displays the accuracy/speed-up heatmap of the 2D early exit in Llama-3.1-8B model in both vanilla version, using the MMS dataset testing split. The axes are the hyperparameters τ_{acc} (accuracy threshold) and τ_{ignore} (ignore threshold). The heatmap show a well-behaved and broad global accuracy maximum, suggesting fast convergence of various hyperparameter search methods. The best speed-up for the 2% accuracy loss is found for $\tau_{acc} = 27.74$, $\tau_{ignore} = 0.4$.

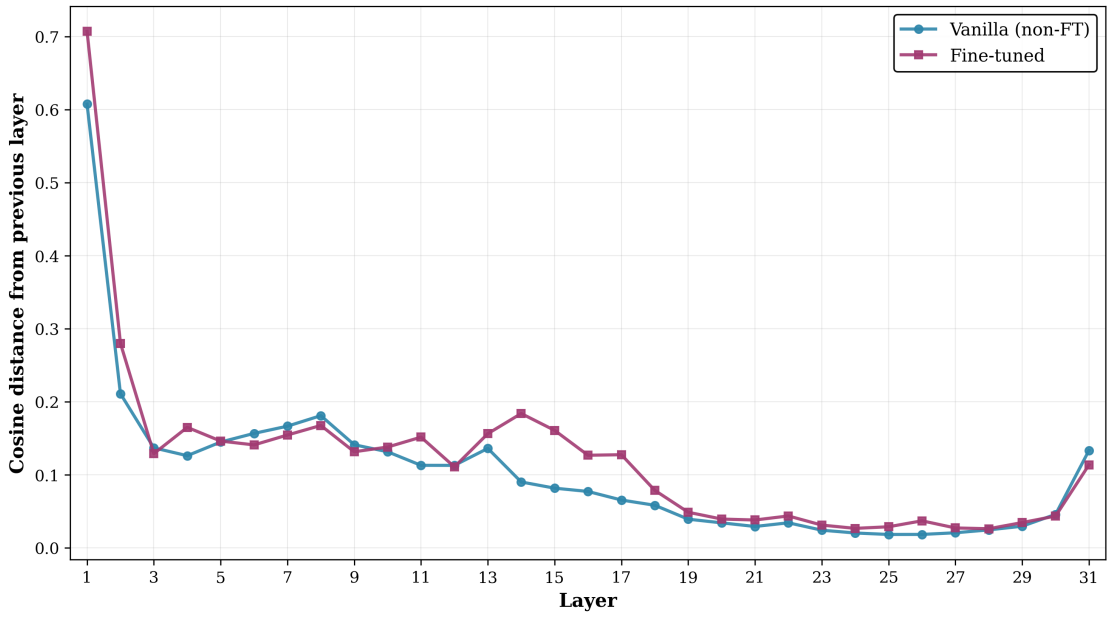

Figure 5: Cosine distances of embedding vectors between consecutive layers of Llama-3.1-8B, averaged over the MMS testing split.

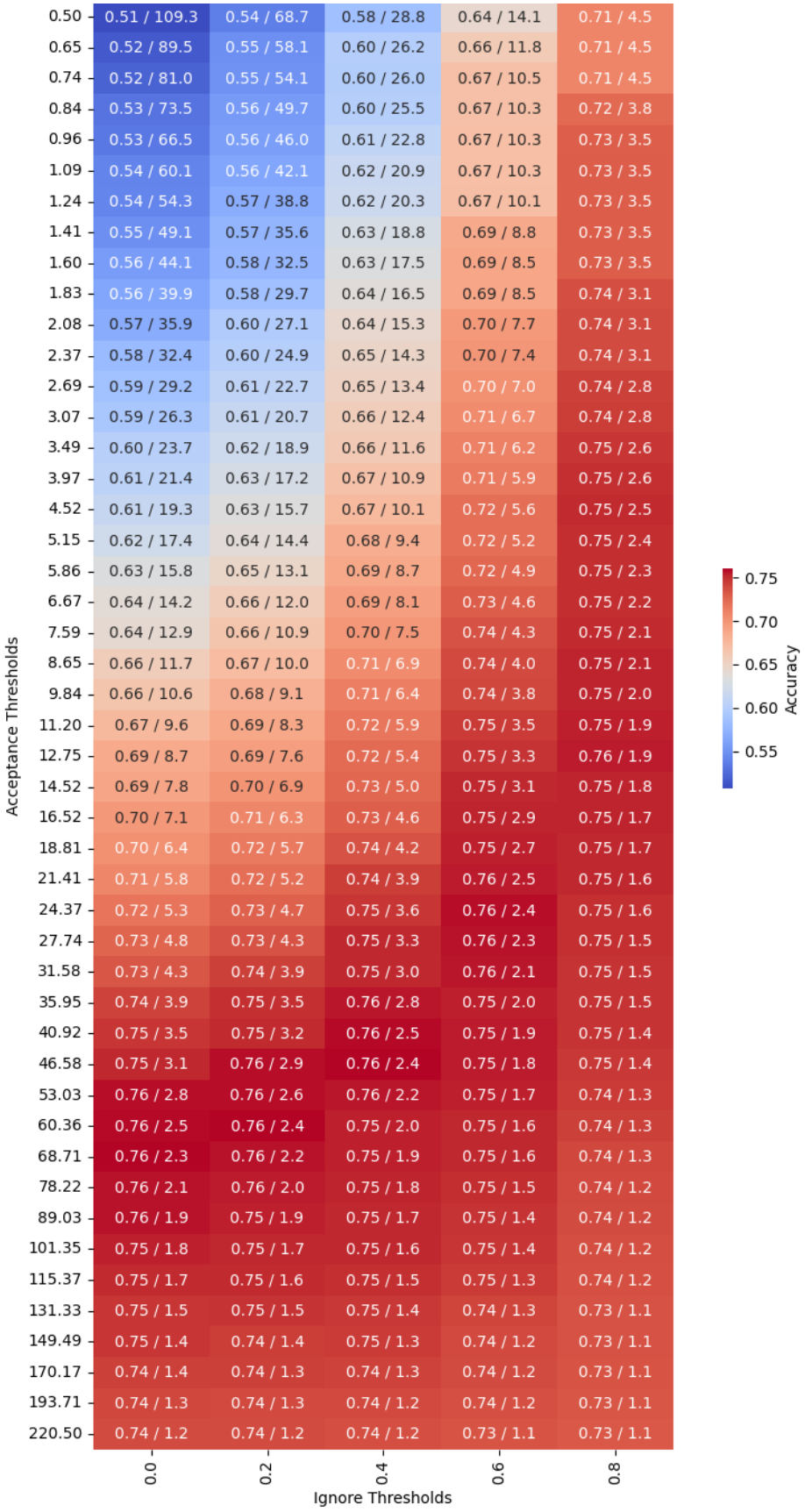

Figure 6: Accuracy and speedup heatmap of the 2D early exit strategy for Llama-3.1-8B on the MMS dataset, no fine-tuning.